

递归

递归：函数多次调用自身

```
void f()  
{  
    f();  
}  
  
int main()  
{  
    f();  
}
```

主函数

f()

f()

f()

死循环

只有自定义函数才可以递归，
主函数不可以递归。

我们可以把递归简单的理解
为一个函数的循环。

程序怎么执行？

递归

递归就是函数多次自己调用自己。只是这个自定义函数完成了一定的功能。

在解决一个大的问题时，可以转化为一个与原问题相似的规模较小的子问题，把处理这个问题的过程写到自定义函数里。

比如 求 $1+2+3+\dots+n$

$$f(k)=f(k-1)+k;$$

递归的三要素

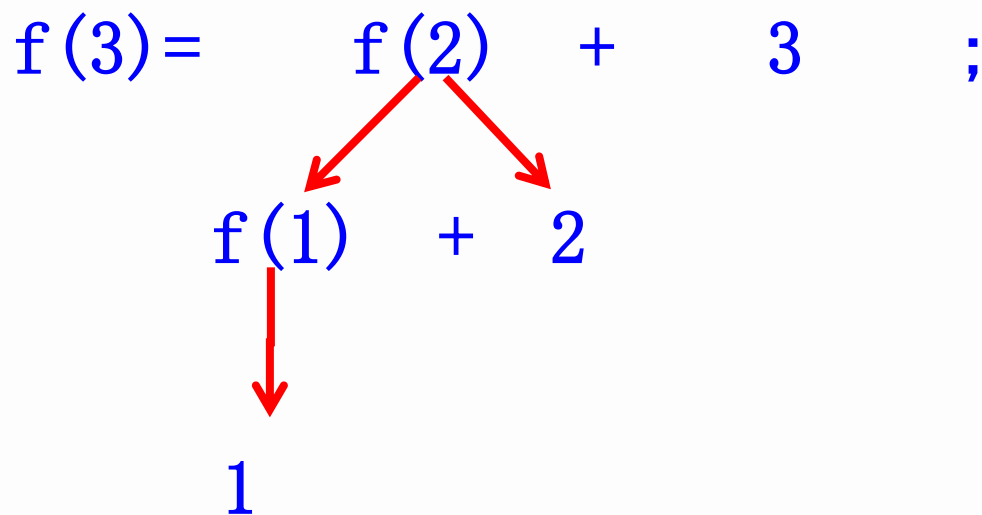
```
int f(int a)
{
    if(a==1)
        return 1;
    return a+f(a-1);
}

int main()
{
    int s=f(3);
    printf("%d", s);
    return 0;
}
```

第一次调用 $f(3)$ 得到
 $f(3)=3+f(2)$;
第二次调用 $f(2)$ 得到
 $f(2)=2+f(1)$;
第三次调用 $f(1)$ 得到
 $f(1)=1$;

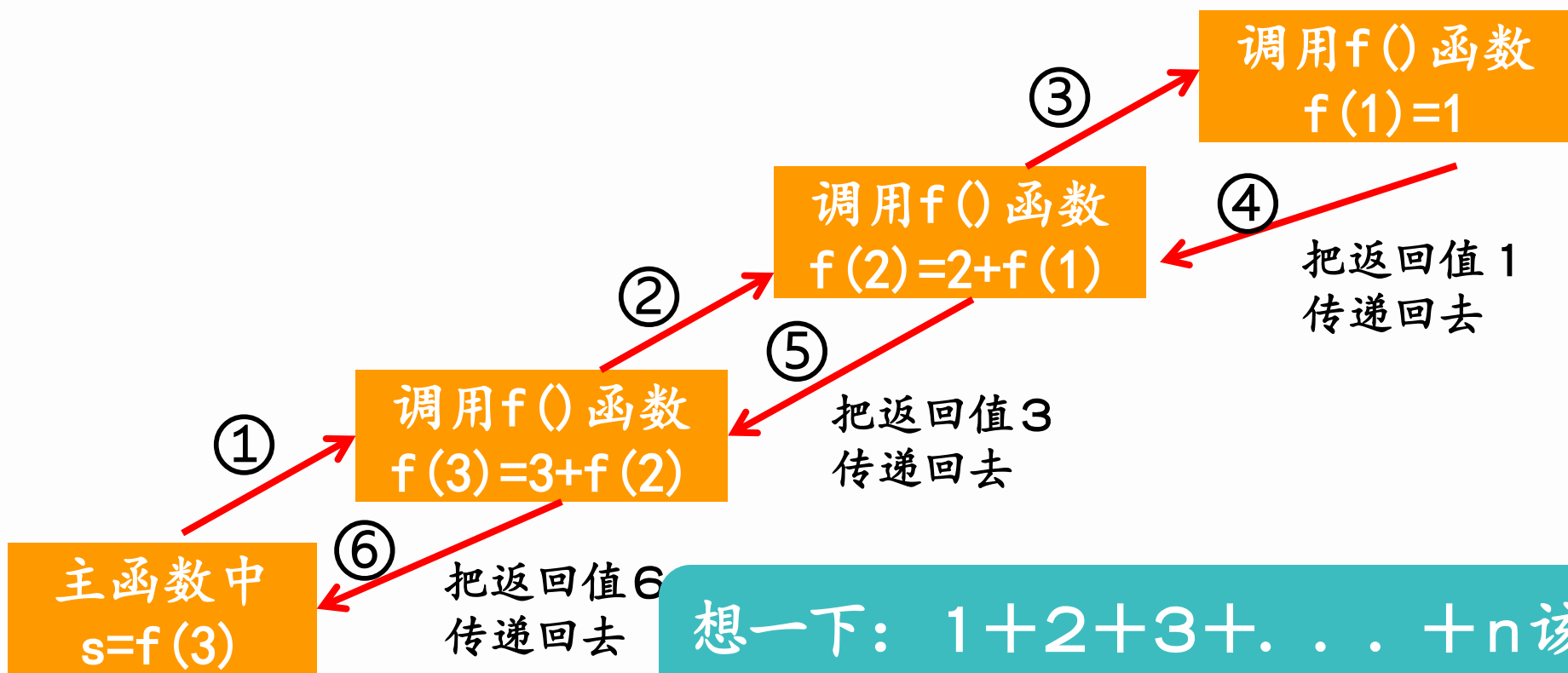
然后再返回调用他的函数
 $f(2)=3$;
 $f(3)=6$;

递归过程



然后把1的值返回给 $f(1)$ ，这样 $f(1)=1$ ；
再把 $f(1)+2$ 的值返回给 $f(2)$ ，这样 $f(2)=3$ ；
最后把 $f(2)+3$ 的值返回给 $f(3)$ ，这样 $f(3)=6$ 。
程序结束

递归的三要素



想一下： $1+2+3+\dots+n$ 该
如何用递归来写

递归的三要素

```
int f(int a)
{
    if(a==1)
        return 1;
    return a+f(a-1);
}

int main()
{
    int n;
    scanf("%d",&n);
    int s=f(n);
    printf("%d",s);
}
```

可以理解为：

设前 n 个数之和为 $f(n)$ ，那么
 $f(n) = f(n-1) + n$;

接下来我们就需要求 $f(n-1)$
是多少就可以了

$f(n-1) = f(n-2) + (n-1)$;
我们会发现每一次的求解方法都是一样的，所以我们可以把这种求解的方法写出一个函数，每次更改函数的参数就可以了。最终 $f(1) = 1$ 表示递归结束。

递归的三要素

一：终止条件

递归一定需要一个终止条件，即满足条件后就不再递归了，这也就和循环一样，当满足循环的终止条件就不在继续循环了，否则程序就是一个死循环(如右边)

二：需要求解的问题可以化成子问题求解，其子问题的求解方式与原问题相同，只是化简到更小的规模

三：父问题与子问题不能有重叠的部分

斐波拉契数列

第一步：先写主函数部分：

```
int main()
{
    int n;
    scanf("%d", &n);
    s=fbi(n);
    printf("%d", s);
    return 0;
}
```

用一个函数 `fbi(n)` 表示求斐波拉契数列的第 `n` 项。并且把返回值的
结果用 `s` 保存。

斐波拉契数列

第二步：先写递归终止条件。

```
if (n==1 || n==2)  
return 1;
```

```
int f(int n)  
{
```

第三步：递归方式。
类似于递推公式

```
return f(n-1)+f(n-2);
```

```
    if (n==1 || n==2)  
        return 1;  
    return f(n-1)+f(n-2)  
}
```

相当于：

```
f(n)=f(n-1)+f(n-2);
```


Diagram illustrating the recursive calculation of $\text{Fibonacci}(3)$. The expression $f(3) + f(2) + f(2) + f(1)$ is shown. Red arrows indicate the flow of computation: $f(3)$ is calculated as $f(2) + f(1)$, which are further broken down into base cases of 1. The final result is 4.

所以 $f(5)$ = 把最后的5个1加起来 = 5

但是我们会发现，相比较递推，递归的计算量比较大，即很多值重复计算，比如 $f(3)$ 。在递归计算 $f(4)$ 的时候，把 $f(3)$, $f(2)$, $f(1)$ 的值都计算出来了，后面再递归 $f(3)$ 的时候，又算了一遍。



斐波拉契数列(优化)

我们已经知道，在使用递推的时候。很多计算都是重复的，这是因为我们在计算的时候计算出一个前面的 $f(k)$ 的值没有存储，后面需要 $f(k)$ 的值得时候，我们又重新算了一遍，很浪费时间。所以我们可以开一个数组记录下来。

```
long long fbi(int i)
{
    if(i==1 || i==2)
    {
        a[i]=1;
        return 1;
    }
    if(a[i]!=0)
        return a[i];
    return a[i]=fbi(i-1)+fbi(i-2);
}
```

记录 $a[1]$ 和 $a[2]$ 的值为1

如果 $a[i]$ 的值已经被记录了，就直接返回该值

记录并且返回 $a[i]$ 的值

递推和递归

递推：也叫顺推。即从已知条件往最终目标逐步推导。

优势：方便快捷，重复计算小，编写简单，难点就是对思维的要求比较大，需要寻找递推公式

递推：也叫逆推。即从最终目标往已知条件逐步推导。

优点：思维难度较低，但是重复计算量比较大，容易理解但是编写较为复杂。

能用递推解决的问题一般都可以用递归来解决，但是能用递归解决的问题不一定可以用递推来解决。如果一个问题可以用递推解决就用递推解决，因为一般递推的时间复杂度较低，代码编写比较简单，不容易出错。

幂运算

已知 a 和 n , 求 a^n 的值对123456取余后结果是多少,
 $a \leq 1000000$, $n \leq 1000$

输入:

2 5

输出:

32

注意:

递归非常耗费时间, 如果没有记忆化搜索的递归, 一般情况下, 递归50+次以上就会爆栈, 你也可以简单的理解为超时, 因为每一次调用函数都会记录原来函数的很多参数。以便调用完之后返回该函数。就算是记忆化搜索后的递归, 一般调用次数也就上千次。多了会爆栈或者超时

幂运算

为什么计算的时候不需要标记

```
long long mi(int i)
{
```

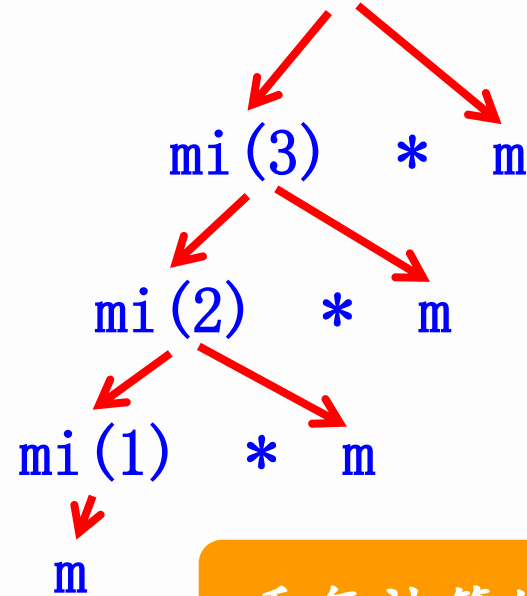
```
    if(i==0)
    {
        a[0]=1;
        return 1;
    }
```

```
    if(i==1)
    {
        a[1]=m%123456;
        return a[1];
    }
```

```
    return a[i]=(mi(i-1)*m)%123456;
```

```
}
```

$mi(5) = mi(4) * m$;



重复计算的只有m

快速幂

已知 a 和 n , 求 a^n 的值对123456取余后结果是多少,
 $a \leq 1000000$, $n \leq 1000000$

输入:

2 5

输出:

32

快速幂:

如果 k 是奇数: $a^k = (a^{(k/2)}) * (a^{(k/2)}) * a$

如果 k 是偶数: $a^k = (a^{(k/2)}) * (a^{(k/2)})$

有的时候, 在程序计算的时候, 为了优化时间复杂度, 我们求幂运算的时候需要用快速幂, 时间复杂度为 $O(\log n)$, 如果快速幂用非递归来写的话非常复杂, 但是用递归就特别简单

快速幂

```
if (i==1)
{
    a[i]=m%123456;
    book[i]=1; //用book数组标记, 以免余数为0
    return a[i];
}
if (book[i] != 0)
    return a[i];
if (i%2==1)
{
    book[i]=1;
    return a[i] = ((mi(i/2)*mi(i/2))%123456)*m%123456;
}
else
{
    book[i]=1;
    return a[i] = (mi(i/2)*mi(i/2))%123456;
}
```

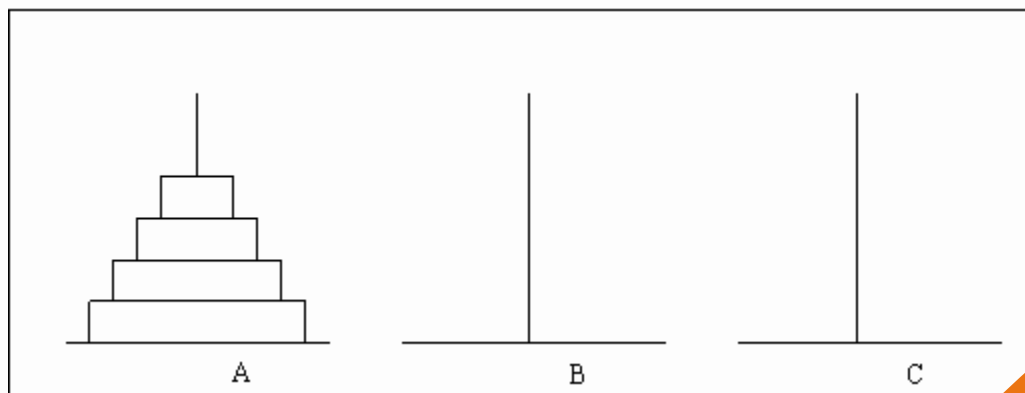
如果以前已经计算过

如果是奇数

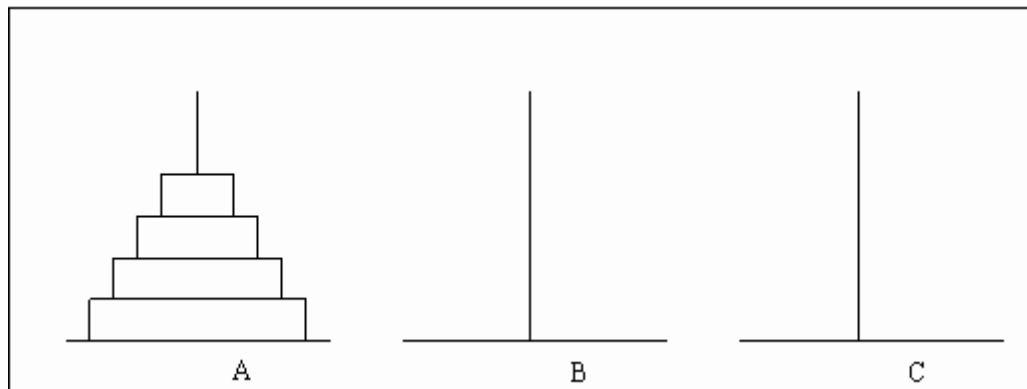
如果是偶数

汉诺塔

法国数学家爱德华·卢卡斯曾编写过一个印度的古老传说：在世界中心贝拿勒斯（在印度北部）的圣庙里，一块黄铜板上插着三根宝石针。印度教的主神梵天在创造世界的时候，在其中一根针上从下到上地穿好了由大到小的64片金片，这就是所谓的汉诺塔。不论白天黑夜，总有一个僧侣在按照下面的法则移动这些金片：一次只移动一片，不管在哪根针上，小片必须在大片上面。僧侣们预言，当所有的金片都从梵天穿好的那根针上移到另外一根针上时，世界就将在一声霹雳中消灭，而梵塔、庙宇和众生也都将同归于尽。



汉诺塔



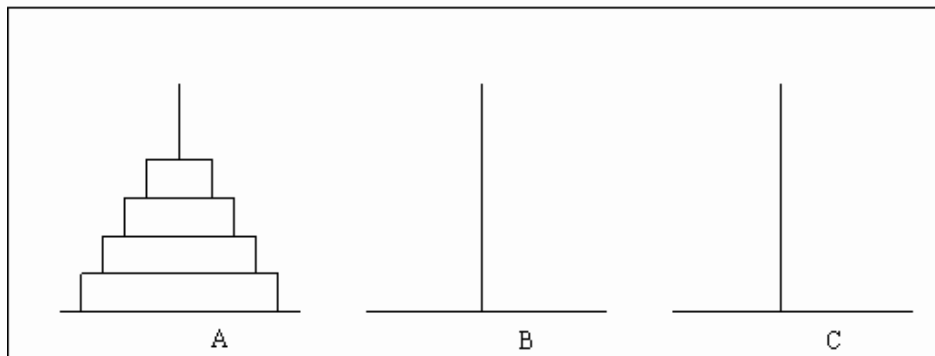
问题分析：

如果我要把柱子A上的 n 个环借助柱子B移动到C上，那么我需要先把A上的 $(n-1)$ 个环借助柱子C移动到B上，再把剩下的一个环直接从A移动到C上，这样最下面那个环就移动好了。然后再把柱子B上面的 $(n-1)$ 个环通过柱子A移动到柱子C上面。这样我们就完成了。

至于我们怎么移动上面 $(n-1)$ 个环，是不是直接重复前面的操作就可以了，把 $(n-1)$ 个环分成上面 $(n-2)$ 个环和最下面一个环。当分到不能自分的时候就结束了。

是不是很神奇！！！！

汉诺塔



每次移动的时候都把当前状态下的上面 $(n-1)$ 个环看成一个整体，下面一个环看成一个整体。

```
void mov(int n,char a,char b,char c)
{
    if(n==0)
        return ;
    mov(n-1,a,c,b); //把前(n-1)个环从柱子a通过柱子c移动到柱子b
    printf("%d %c->%c\n",n,a,c); //把第n个环从a移动到c。
    mov(n-1,b,a,c); //再把前(n-1)个环从柱子b通过柱子a移动到柱子c
    //这样操作之后就移动好了。
}

mov(n,'a','b','c'); //把n个环从柱子a通过柱子b移动到柱子c
```

递归

递归除了用来表示从未知目标状态到已知初始状态的一个转移。还可以用来表示重复执行某一系列的操作。

我们可以把一系列重复的操作封装成一个函数，然后每次都重复执行这个函数。这种操作就叫做递归。也可以理解为函数的循环。和普通循环一样，递归也必须要有初始状态(递归的起点)，变化增量(下一次递归的状态)，终止条件(递归的终点)。三者缺一不可。

全排列

有1--n这样n个数，把这n个数组合成一个数字，求这样的组合有多少种

输入：

3

输出：

6

(123 132
213 231
312 321)

遇到这样的问题，我们首先考虑到的就是排列组合问题，即n个数的全排列就是 A_n^n ，即 $3*2*1=6$ 。但是有的时候，可能题目中有一些限制，比如第i个位置不可以放置数字k。这样全排列公式就不好用了。我们就要换一种思路考虑

源代码

```
void dfs(int step)//表示当前我站的位置
{
    if(step==n+1)//如果前面n个盒子都放置好了
    {
        N++;//方案数+1(全局变量)
        return ;
    }
    for(int i=1;i<=n;i++)//从小到大遍历所有可能
    {
        if(book[i]==0)//如果该数字还未被放置
        {
            book[i]=1;//放置进该盒子，并且标记
            dfs(step+1);//按照这种方法放置下一个盒子
            book[i]=0;//回溯，取消标记。
        }
    }
    return ;
}
```

配合后面的详细解释解读代码

全排列

解题思路：

我们可以把这道题看成是把 n 个数字放到 n 个盒子里面去。



首先，我们来到了第一个盒子的位置，此时我们需要放一个数字进去。可以放该数字的条件是：

- (1) 这个数字是在这个范围之内
- (2) 我们手中还有这个数字

我们这里先放入数字1

全排列

1

然后第一个盒子放置完之后，我们就来到了第二个盒子的位置，我们是不是考虑的问题和刚才一样？

(1) 这个数字是在这个范围之内

(2) 我们手中还有这个数字

由于1已经放置过了，我们手中只剩下2--n了，所以我们先放入2，我们怎么才可以知道这个数字是否可以放呢？很简单，开一个数组记录一下就可以了。

我们这里放入数字2

全排列

1

2

3

4

5

同样的，后面3--n个盒子的考虑方法和前面的考虑方法一样，我们最开始的时候依次放入的就是3，4，5。。。n。

前面我们已经知道，递归肯定是有结束条件的，结束条件是什么呢？当我们走到第n+1个盒子的时候，是不是前面n个盒子都放了数字了？那我手上肯定就没有数字可以放置了，所以放置游戏就结束了。

但是1 2 3 4 5只是我的一种放置方法，而我是要求出所有的放置方法，该怎么办呢？我只是手中已经没有数字了。很简单，把最后一个位置的数字(5)拿起来，看一下还有没有其他数字可以放置。很显然没有。

全排列

1

2

3

4

那我们该怎么办呢？聪明的同学已经想到了，我再退回上一格，即第4个格子，把第四个格子的数字拿出来，此时我手上就有4,5两个数字了，注意，前面我已经把数字5 拿出来了，那么说明数字5没有放入进去，所以取消标记。上一次放的4，那么我这次就放5了，再继续往下一个盒子走，然后在第五个位置放入4. 这就是第二种放方法。我们发现又放完了，继续退回上一个格子，如果上一个格子的所有方案都放置过了后，我们继续退回再上一个格子，继续前面的操作，直到第一个格子的所有方案数都放置完之后就结束了。

借书问题

学校放暑假时，信息学辅导教师有 n 本书要分给参加培训的 n 个学生。如A, B, C, D, E共5本书要分给参加培训的张、刘、王、李、孙5位学生，每人只能选1本。教师事先让每个人将自己喜爱的书填写在如下的表中，然后根据他们填写的表来分配书本，希望设计一个程序帮助教师求出可能的分配方案，使每个学生都满意。

	A	B	C	D	E
张			Y	Y	
王	Y	Y			
刘		Y	Y		
孙				Y	
李		Y			Y

借书问题

输入格式第一行一个数 n （学生的个数，书的数量）

以下共 n 行，每行 n 个0或1（由空格隔开），第 i 行数据表示第 i 个同学对所有书的喜爱情况。0表示不喜欢该书，1表示喜欢该书。

输出格式依次输出每个学生分到的书号。

样例

输入book. in

5

0 0 1 1 0

1 1 0 1 1

0 1 1 0 0

0 0 0 1 0

0 1 0 0 1

输出book. out

1

3 1 2 4 5

第一行输出方案总数

以下每行输出一个可行方案、每个数字
间用空格隔开

（按照字典序最小顺序输出）

借书问题

问题分析：

我们分析下第 i 个同学可以借某本书需要的条件：

- (1) 这本书是否存在
- (2) 第 i 个同学是否喜欢这本书
- (3) 这本书是否已经在前面被借走了。

是不是这就是需要的所有条件了呢？

并不是，我们还需要判断，在该种情况下，其他所有的同学是否都借到了让自己满意的书

借书问题

```
void dfs(int k)//表示当前第k个学生借书
{
    if(k==n+1)//如果前k个同学都成功借到书了
    {
        for(int i=1;i<=n;i++)
            printf("%d ",b[i]);
        printf("\n");
    }
    for(int i=1;i<=n;i++)//所有书的范围
    {
        if(a[k][i]==0||book[i]==1)//如果不喜欢或者如果被借了
            continue;
        book[i]=1;//表示借这本书，这本书就不能被其他人借走了
        b[k]=i;//记录第k个人借了第i本书
        dfs(k+1);//考虑下一位同学
        book[i]=0;//考虑其他情况
        b[k]=0;//考虑其他情况
    }
}
```

递归模板

```
void dfs(int i); 查询当前(第i种)位置
```

```
{
```

```
    if (i==n +1) //表示前n个都考虑好了  
        输出结果
```

```
    for( ; ; ) //枚举可能的范围  
    {
```

```
        if (找到一种可行解，不冲突)  
        {
```

```
            记录满足条件的 $X_i$ ;
```

```
            添加相应的标志;
```

```
            dfs(i+1) //查找下一个位置
```

```
            标志还原，考虑其他情况;
```

```
        }
```

```
    }
```

```
}
```



借书问题

